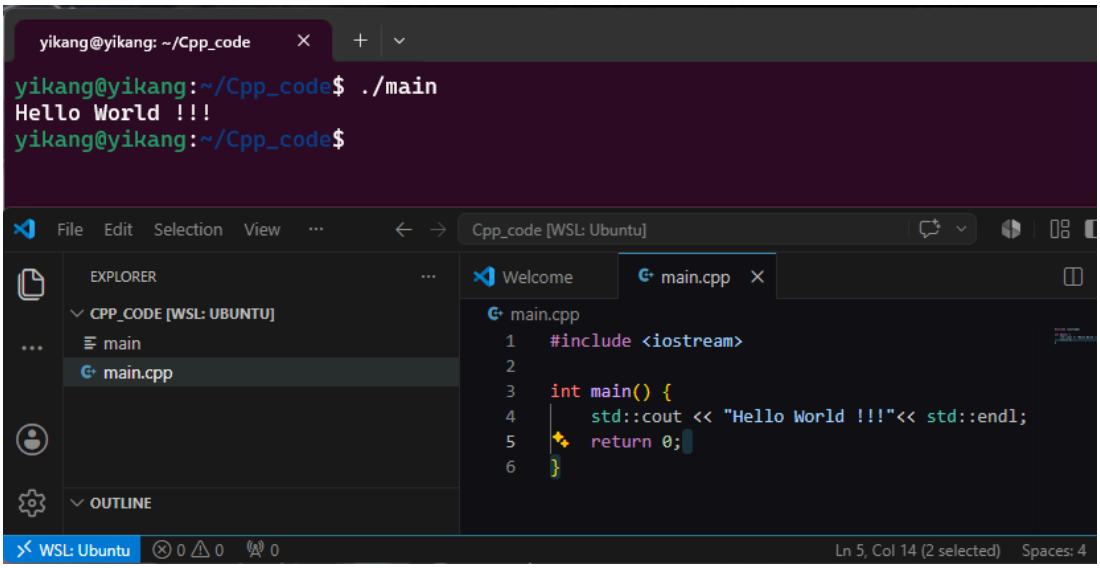
C++ Study

Chapter 2: The Program Skeleton of C++

This chapter mainly introduces the basic structure of a C++ program.



Program Output

This program outputs: Hello world !!!

Reminder: if you know a little about ASCII, you can see that uppercase and lowercase letters have different binary codes.

Therefore, cout must be written as cout, not Cout or COUT. Otherwise, errors will occur.

Supplement: More precisely, the real reason is that C++ is a case-sensitive language. ASCII helps explain that uppercase and lowercase letters have different encodings, but the compiler treats cout, Cout, and COUT as three different names.

First: Understanding main()

main() is the entrance of the program.

An abstract way to understand the process is:

When you run a program, the operating system asks: "Where is the entrance of this program?"

C++ answers: "main()"

Second: int main()

This is the function header.

It means:

```
int      this function will finally return an integer
main     the function name is main
()       this function currently receives no parameters
```

Why do we use int main(), instead of something else?

Because the program has this structure:

```
int main() {
    Statement
    return 0;
}
```

You can see that the value returned above is 0, and 0 is an integer value.

At this point, you may have another question:

```
string main() {
    Statement
    return "success";
}
```

At first glance, the program above may look reasonable. But from the computer and operating system point of view, why are you giving it the word "success"?

The operating system normally uses an integer exit code. In many cases, 0 means success. Therefore, int is the return type of main().

Supplement: A more precise wording: int is the return type, while main is the function name. The integer returned by main() is handed back to the operating system as an exit code. Usually, 0 means the program ended successfully.

void main()

If you have studied Java, you may wonder why we do not use void main().

```
This tells the compiler that a function named main exists, returns nothing, and
takes no parameters.
```

However, this is not the standard C++ form for a normal program, even though some compilers may support it.

It is like Alexander Hamilton standardizing currency: everyone uses USD as the payment method. Standard forms prevent confusion.

Supplement: void means "returns no value." But in standard C++, the ordinary program entrance is normally written as int main(), because main returns an integer status code to the operating system.

What is #include <iostream>?

If you have read my OS study notes, you may know that this is a preprocessor directive.

The rough meaning is:

I want to use C++ input and output tools, so please provide the related definitions for me.

What does std mean?

In the C++ standard library, many tools have already been declared.

```
std::cout
std::endl
```

Supplement: Use two colons: std::cout and std::endl. iostream is a header file; std is a namespace; cout and endl are names inside the std namespace.

What is cout?

cout is like the output pipe that leads to the screen.

What is << ?

It is an insertion operator in this context.

```
cout << "Hello";
```

It inserts the thing on the right into the output stream on the left.

cout is like a conveyor belt.

<< is the action of putting something onto the conveyor belt.

"Hello" is the package you put onto the belt.

The screen is the exit of the conveyor belt.

But this same symbol can mean something else:

```
8 << 1
```

Here it means a binary left shift.

```
x << n is roughly x * 2^n
```

Therefore, you must understand the symbol according to the context.

Supplement: More accurately, 8 << 1 shifts the binary form of 8 left by one bit. 8 is 1000 in binary. After shifting left, it becomes 10000 in binary, which is 16 in decimal. For non-negative integers without overflow, x << n is roughly x * 2^n.

What is endl?

std::endl means newline, and it also flushes the output buffer. At the beginner level, you can first understand it as "go to a new line."

Supplement: cout does not automatically move to a new line. Without std::endl or "\n", the next cout output immediately follows the previous output.

```
cout << "Hello\n";
cout << "Hello" << endl;
```

"\n" is the newline character. Its main job is to start a new line. std::endl also starts a new line, but it additionally flushes the output buffer. At this stage, both can be understood as newline tools, but endl does one extra flush action.

Tokens and Source Code Formatting

C++ understands code through tokens. For example:

```
int main()
{

}
```

The important tokens include int, main, (,), {, and }. White space separates tokens.

Supplement: Token means an indivisible unit in code, such as int, main, (,), {, }. Spaces, tabs, and newlines are called white space. They help separate tokens.

C++ Source Code Formatting

C++ is a free-format language. The compiler mainly cares about tokens and semicolons. It usually does not care much about your line breaks or indentation.

```
int main() { return 0; }

int main ( ) { return 0; }  // also valid

int ma in() { return 0; }   // wrong: main is broken apart
re
turn 0;                     // wrong: return is broken apart
```

So formatting is flexible, but you cannot break a keyword or a name. Good C++ style usually puts one statement on one line and indents statements inside a function body.

What is cin?

cin is the standard input object.

For example:

```
cin >> carrots;
```

This reads a value from the keyboard and stores it into the variable carrots.

cin is convenient because it looks at the type of the variable that has already been declared. The computer receives keyboard input as characters, but cin converts the input into the declared variable type when possible.

Supplement: cin >> carrots; can be visualized as keyboard data flowing into the variable carrots.

Variable Declaration Statement

```
int carrots;
```

int means this variable stores an integer. carrots is the variable name. This statement prepares a memory location that can store an integer and labels that location carrots.

Assignment Statement

```
carrots = 25;
carrots = carrots - 1;
```

In C++, = is not a mathematical equal sign. It is the assignment operator. It means: put the value on the right into the variable on the left.

carrots = carrots - 1; is not a mathematical equation. It is a state update. First, compute the right side; then store the result back into the left side.

Variable Name vs String

```
cout << carrots;     // outputs the value stored in carrots
cout << "carrots";   // outputs the word carrots
```

carrots without quotation marks is a variable. "carrots" with quotation marks is a string.

Function Prototype

Before the compiler uses a function, it needs to know:

- What the function is called
- What parameters it needs
- What type it returns

```
double sqrt(double);
```

In this example, sqrt is a function.

It needs one double argument.

It returns a double result.

Argument and Return Value

```
x = sqrt(6.25);
```

6.25 is the argument: the information sent into the function. sqrt(6.25) computes the square root and returns 2.5. That 2.5 is the return value. Finally, 2.5 is assigned to x.

Simple memory rule: argument is what you send in; return value is what the function sends back.

Function Definition

A function prototype and a function definition are different. For sqrt(), we did not write the function ourselves; it comes from the library.

Now we can define our own function:

```
double square(double x) {
    return x * x;
}
```

In this example, we define a function called square.

This function receives a double parameter.

It returns a double result.

Some function prototypes come from other libraries. Therefore, we need to tell the compiler where the related declarations are located.

```
#include <cmath>
```

By including <cmath>, we tell the compiler that the declaration for sqrt() is available through the cmath header. During the build process, the compiler and linker can then handle the function correctly.

main.cpp > main()

```
1   #include <iostream>
2   #include<cmath>
3
4   int main(){
5
6       using namespace std;
7       double area;
8       cout << "Enter the floor area: ";
9       cin >> area;
10
11      double side;
12      side = sqrt(area);
13      cout << "That is the equal to square "
14           << side << "feet on each side." << endl;
15
16      return 0;
17  }
```

double area and double side

```
double area;
double side;
```

```
cin >> area;
side = sqrt(area);
```

area is a variable name we define ourselves. It stores the area. side is also a variable name we define ourselves. It stores the side length. double means the variable can store decimal values.

side = sqrt(area); means: take the square root of area and store the result in side. The mathematical relationship is: area = side * side, so side = sqrt(area).

If you understand this part, you can continue reading the chapter.

Now We Start Understanding void

```cpp
#include <iostream>

void simon(int);
int main(){

    using namespace std;

    simon(3);
    return 0;
}

void simon(int n){
    using namespace std;
    cout << "Simon says touch your toes " << n << "  times." << endl;

}
```

The first void in void simon(int); tells the compiler that this is a function prototype for a function called simon.

Inside main(), we call the simon function and pass 3 into it.

simon(3); passes the argument 3 to the simon function. When execution enters the function definition void simon(int n), the parameter n receives this value, so inside the function n = 3.

Prototype / Call / Definition Comparison

```
void simon(int);        // function prototype: early registration
simon(3);               // function call: actually call the function
void simon(int n)       // function definition: the real function body
{
    cout << n << endl;
}
```

Execution Order of simon

main() and simon() do not run at the same time. The program starts in main(). When main() meets simon(3);, it jumps to the simon function. The value 3 is passed to n, so n = 3. After simon finishes, the program returns to main() and continues with return 0.

```
main() starts
|
meets simon(3)
|
3 is passed to n
|
runs the body of simon
|
returns to main()
|
return 0
```

A Function with a Return Value: stonetolb

A void function only does an action and returns no result. An int function returns an integer result. The job of stonetolb is to convert stones to pounds. 1 stone = 14 pounds.

```
#include <iostream>
using namespace std;

int stonetolb(int);

int main()
{
    int stone;
    cout << "Enter the weight in stone: ";
    cin >> stone;

    int pounds = stonetolb(stone);

    cout << stone << " stone = ";
    cout << pounds << " pounds." << endl;
    return 0;
}

int stonetolb(int sts)
{
    return 14 * sts;
}
```

int stonetolb(int); is the function prototype. It tells the compiler that stonetolb receives an int and returns an int.

int pounds = stonetolb(stone); is the function call. The value of stone is passed to sts.

return 14 * sts; returns the calculation result to main(). If stone = 15, then sts = 15, and the function returns 14 * 15 = 210.

Chapter 2 Mini Summary

The common statement types in this chapter can be remembered like this:

```
int carrots;              // declaration statement
carrots = 25;             // assignment statement
cout << "Hello";          // message statement
simon(3);                 // function call
void simon(int);          // function prototype
return 0;                 // return statement
```

Core understanding: a C++ program starts from main(); variables let the program store data; cin and cout let the program input and output data; functions let the program be divided into smaller modules; an argument is information sent into a function; a return value is information sent back from a function.